

Distributed Object-Recognition System with AI and the Raft Consensus Algorithm

CC4P1 – Concurrent and Distributed Programming – Final 2026-I

Melissa Iman Noriega Junior Ortega

School of Computer Science – Faculty of Sciences – UNI

2026-I

- 1 Problem and requirements
- 2 Architecture
- 3 Raft consensus
- 4 AI model
- 5 Results and demo

Goal: a distributed system that recognizes objects (or animals/people) with a previously trained AI model, and stays consistent and fault-tolerant using the **Raft consensus algorithm**.

- **Training Server** – trains the model and its weights; the load can be split across nodes (CPU-parallel). Weights are persisted.
- **Testing Server** – each camera uses the trained model, recognizes objects on its own, stores an image and records the detection.
- **Watcher Client** – shows the consistent detection log (type, camera, date/time).
- **Raft Consensus Module** – replicated log + state machine; if a node fails, the others keep going.

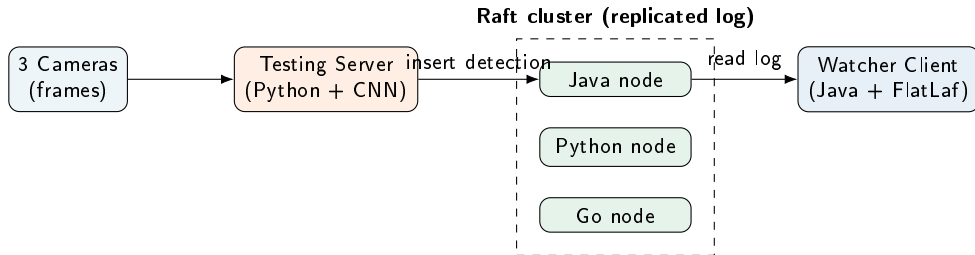
Hard constraints from the assignment

- **Raw native sockets only.** No WebSocket, Socket.IO, frameworks, RabbitMQ/MQ, or communication libraries. **Raft is implemented by hand over TCP.**
- **Three languages**, one node per student: **Java, Python, Go.**
- **Two operating systems** running the same set of languages.
- **Threads** for performance and to avoid record corruption.
- **Local deployment, no internet**, on LAN and WiFi.
- Only the base libraries of each language (NumPy allowed for the CNN).

Member	Language	Component
Melissa Iman	Java	Raft core + distributed training + Watcher UI
Junior Ortega	Go	Raft node + record state machine
(Python part)	Python	CNN + Testing Server + Raft node

All three nodes speak the **same text protocol**, so they form one heterogeneous cluster.

System architecture

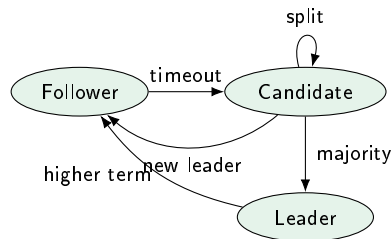


- Cameras feed frames to the Testing Server; the CNN recognizes and inserts each detection into the cluster **through the leader**.
- The Watcher reads the committed, replicated log.

Raft: leader election and log replication

States: Follower / Candidate / Leader.

- Randomized election timeout **150–300 ms** (avoids split votes).
- RequestVote with the election restriction (candidate log must be up to date).
- AppendEntries replicates; an entry is **committed** once it is on a **majority**.
- Log Matching + nextIndex to repair followers.
- Only current-term entries commit by counting replicas.



The text protocol (shared by the 3 languages)

One message per line, fields separated by |:

```
REQUEST_VOTE|term|candidateId|lastLogIndex|lastLogTerm
VOTE|term|granted(1/0)
APPEND|term|leaderId|prevLogIndex|prevLogTerm|commitIndex|entries
APPEND_OK|term|success(1/0)|matchIndex

NUEVA_DETECCION|type,imageRef,dateTime,camera -> insert a detection
LEER_REGISTRO -> read the whole log
REGISTRO|det1;det2;det3;... -> reply to the Watcher
REDIRECT|leaderHost:leaderPort -> if a follower was contacted
OK -> insertion accepted
```

Because Java, Python and Go all use this exact format, a node in any language interoperates with the others.

The CNN (from scratch with NumPy)

Architecture (5 classes of figures):

input $1 \times 28 \times 28$
↓ Conv $8@3 \times 3$ + ReLU + MaxPool
↓ Conv $16@3 \times 3$ + ReLU + MaxPool
↓ Flatten + Dense + Softmax

- Full forward + backward (real backprop), convolution via `im2col`.
- No TensorFlow/PyTorch – only NumPy, works offline.
- Weights persisted to disk (`.npz`).

Distributed training

- Data-parallel: each mini-batch is split into shards, one worker per shard, gradients are averaged (all-reduce).
- multiprocessing (real CPU parallelism, avoids the GIL).

97.2% test accuracy
same result, faster (4.7s vs 11s)

Verified with the three languages running together:

- Java + Python + Go elect a leader among themselves and replicate.
- The Testing Server runs the CNN on 3 cameras (one thread each) and inserts detections into the cluster.
- **All three nodes apply the same 75 detections in the same order.**

	Java	Python	Go
applied entries	75	75	75

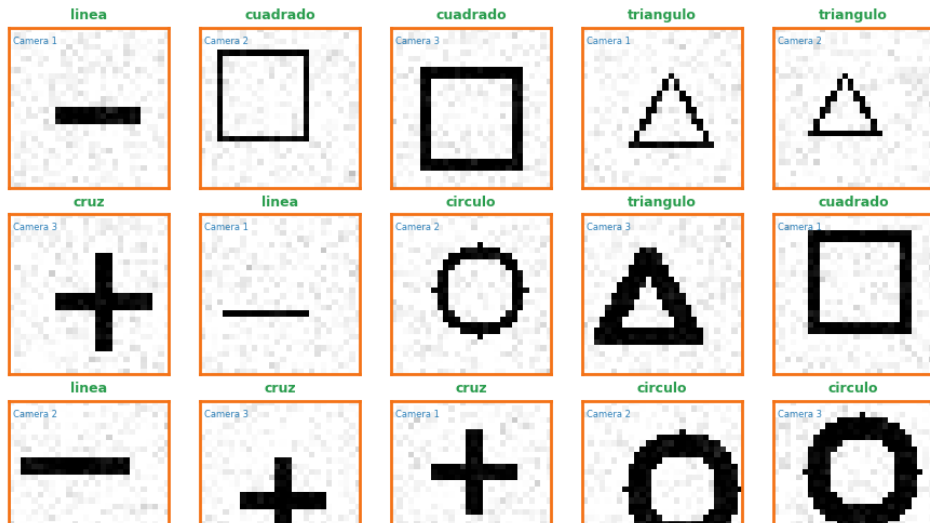
Fault tolerance (leader failover)

- With Python as leader, we kill it mid-run.
- **Go is re-elected** leader in the next term.
- Previous detections survive: the log stays consistent.
- Inserting through the new leader keeps working.

"If a node fails, the others keep going and the record stays consistent." – exactly what the assignment asks for.

Recognition – real detections from the CNN

AI recognition detections



**[screenshot: Watcher UI showing the live detection log
(type, camera, date/time) while the cluster runs]**

The Watcher reads the committed log from the leader and refreshes every second.

Compliance with the constraints

Requirement	Status
Raw native sockets only (no frameworks/MQ)	Done – Raft by hand over TCP
Three languages, one node each	Done – Java / Python / Go
Threads for performance / no corruption	Done – threads, goroutines, mutex
Previously trained AI + persisted weights	Done – CNN 97.2%, .npz
Distributed CPU-parallel training	Done – multiprocessing all-reduce
≥ 3 cameras	Done – 3 camera threads
Fault tolerance (Raft)	Done – verified failover
LAN/WiFi, 2 OS	Pending for the live demo

Thank you

Distributed Object-Recognition System with AI and Raft
CC4P1 – 2026-I